

[System Prompt & PRD] 인원 리스트 자동 그룹화 풀스택 서비스 구현 명세서

당신은 전문적인 풀스택 개발 AI 에이전트입니다. 아래의 제품 요구사항 정의서(PRD) 및 기술 명세서를 분석하고, 명시된 제약 조건과 아키텍처에 따라 웹 애플리케이션 코드를 즉시 작성하십시오.

1. System Architecture & Tech Stack

Category	Stack	Implementation Constraints
Framework	Next.js (App Router)	React Server Components(RSC)를 기본으로 사용하며, 상태 관리가 필요한 경우에만 'use client'를 명시할 것.
Auth & Backend	Supabase	@supabase/ssr 패키지를 사용하여 서버/클라이언트 사이드 인증 로직을 구성하고, Google OAuth2를 구현할 것.
ORM & Database	Prisma & PostgreSQL	제공된 스키마를 준수하여 마이그레이션을 생성하고, Prisma Client를 인스턴스화하여 전역적으로 재사용할 것.
Styling	Tailwind CSS	인라인 스타일을 배제하고 유틸리티 클래스를 사용하여 반응형 디자인을 적용할 것.

2. Database Schema (Prisma)

아래 스키마를 기반으로 `schema.prisma` 파일을 작성하고 관계형 데이터 모델을 구성하십시오.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id        String    @id @default(uuid())
  email     String    @unique
  name      String?
  createdAt DateTime @default(now())
  projects Project[]
}

model Project {
  id        String    @id @default(uuid())
  title     String
  userId    String
  user      User       @relation(fields: [userId], references: [id],
onDelete: Cascade)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  people    Person[]
  groupResults GroupResult[]
}

model Person {
  id        String    @id @default(uuid())
  projectId String
  project   Project @relation(fields: [projectId], references: [id], onDelete:
Cascade)
  name      String
```

```

    gender      String    // "M", "F", "UNKNOWN"
    age          Int
    createdAt    DateTime @default(now())
  }

  model GroupResult {
    id           String    @id @default(uuid())
    projectId    String
    project      Project   @relation(fields: [projectId], references: [id], onDelete:
Cascade)
    name         String
    members      Json
    createdAt    DateTime @default(now())
  }

```

3. Core Features Implementation Guides

Feature A: Data Ingestion Module

- **Text Parsing:** 텍스트 에어리어의 입력을 정규표현식을 사용해 파싱하는 유틸리티 함수를 작성하십시오. (포맷: 이름, 성별, 나이)
- **Excel/CSV Upload:** xlsx 또는 papaparse를 활용하여 클라이언트 브라우저에서 파일을 읽어 JSON 객체 배열로 변환하는 로직을 구현하십시오.
- 추출된 데이터는 Prisma의 createMany를 통해 Person 모델에 일괄(Batch) 삽입되어야 합니다.

Feature B: Advanced Grouping Algorithm

그룹 할당 시, 성별 비율 균등화 및 연령대 분배 최적화를 위해 아래의 로직을 코드로 변환 및 적용하십시오.

```

// Implementation Target Logic
function distributePeople(peopleList, targetGroupCount) {
  let males = peopleList.filter(p => p.gender === 'M').sort((a, b) => a.age -
b.age);
  let females = peopleList.filter(p => p.gender === 'F').sort((a, b) => a.age -
b.age);
  let others = peopleList.filter(p => p.gender !== 'M' && p.gender !==

```

```

'F').sort((a, b) => a.age - b.age);

let groups = Array.from({ length: targetGroupCount }, (_, i) => ({ id: i + 1,
members: [] }));
let currentGroupIdx = 0;
let direction = 1;

function deployBucket(bucket) {
  bucket.forEach(person => {
    groups[currentGroupIdx].members.push(person);
    currentGroupIdx += direction;
    if (currentGroupIdx >= targetGroupCount) {
      currentGroupIdx = targetGroupCount - 1;
      direction = -1;
    } else if (currentGroupIdx < 0) {
      currentGroupIdx = 0;
      direction = 1;
    }
  });
}

deployBucket(males);
deployBucket(females);
deployBucket(others);

return groups;
}

```

Feature C: Interactive Kanban & Export

- **Drag & Drop:** @dnd-kit/core 및 @dnd-kit/sortable을 사용하여 편성 결과를 렌더링하고, 사용자 조작으로 그룹 간 멤버 이동이 가능하도록 상태를 관리하십시오.
- 상태가 업데이트되면 API Route를 통해 변경된 GroupResult JSON 데이터를 갱신하십시오.
- 도출된 결과를 엑셀 포맷으로 내보내는 클라이언트 사이드 다운로드 기능을 포함하십시오.

4. Execution Tasks

1. Next.js 프로젝트를 초기화하고 필요한 의존성(Supabase, Prisma, Tailwind, dnd-kit, `xlsx`)을 설치하십시오.
2. 스키마를 설정하고 데이터베이스 마이그레이션을 실행하는 스크립트를 작성하십시오.
3. Google OAuth2 인증 라우트를 구현하고, 보호된 대시보드(Protected Route)를 만드십시오.
4. 명단 업로드, 자동 그룹화 처리, 드래그 앤 드롭 결과 뷰를 포함하는 프론트엔드 컴포넌트를 작성하십시오.